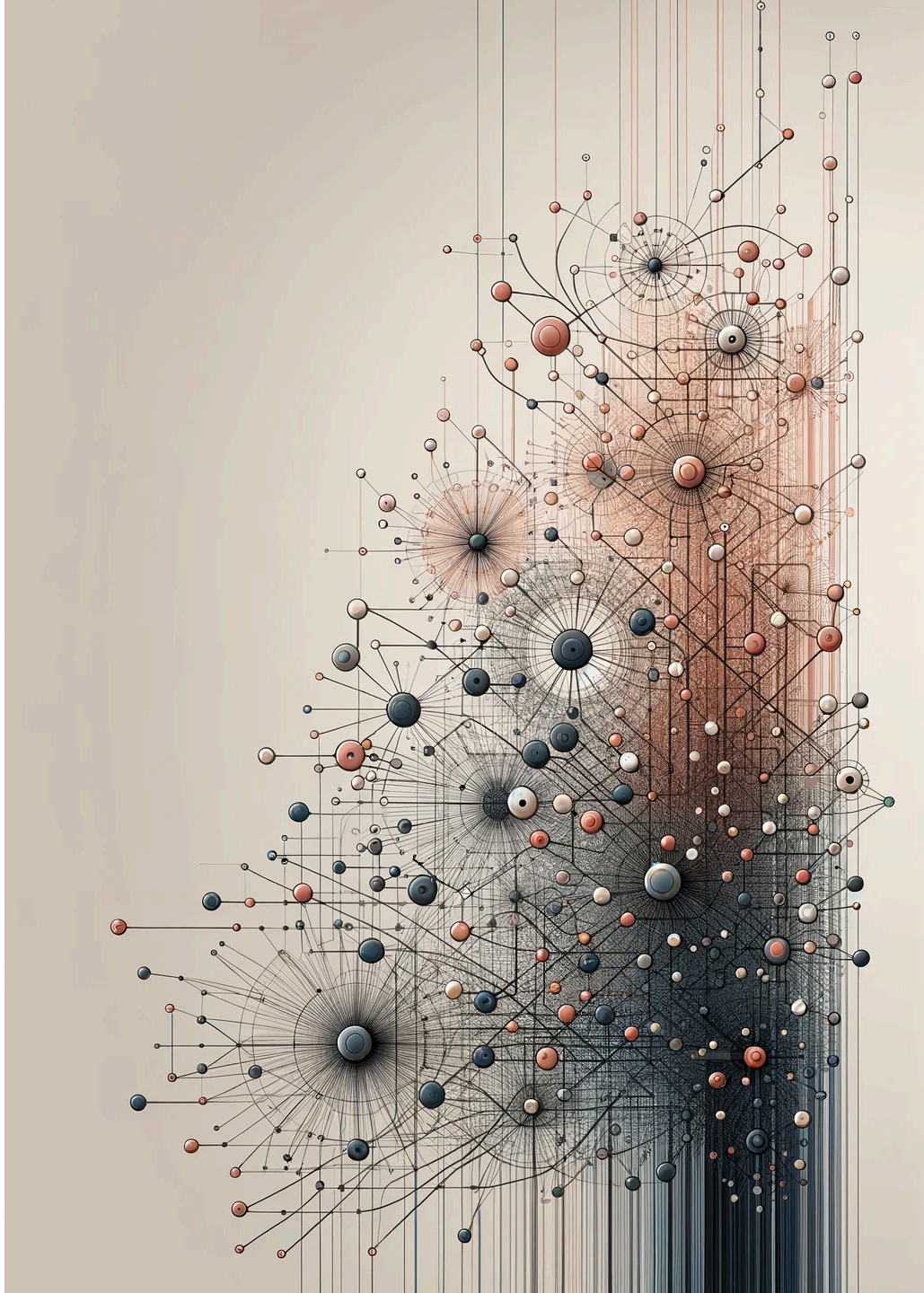
UNIVERSITÀ DI TORINO

Analisi e Visualizzazione delle Reti Complesse

NS12 - Communities

Prof. Rossano Schifanella



Outline

- From node-level to mesoscopic structure
- Why communities matter
- What a community is, and why this is hard to define
- Community variables and basic definitions
- Partitions, overlap, and hierarchy
- Communities versus graph partitioning and clustering
- A map of method families for the next lectures

From NS11 to NS12

- In `NS11` we focused on **global heterogeneity**, rich-get-richer effects, and long tails
- Today we move to a different structural scale:
  - not the whole network
  - not the single node
  - but the **mesoscopic level**
- The guiding question becomes:
  - can we identify groups of nodes that behave as coherent subnetworks?

Three structural scales

- **Microscopic scale**:
  - nodes, edges, local motifs
- **Mesoscopic scale**:
  - communities, modules, blocks, groups
- **Macroscopic scale**:
  - degree distributions, giant components, assortativity, robustness

Community detection is one of the central tools for studying the **mesoscopic organization** of a network.

Why mesoscopic structure matters

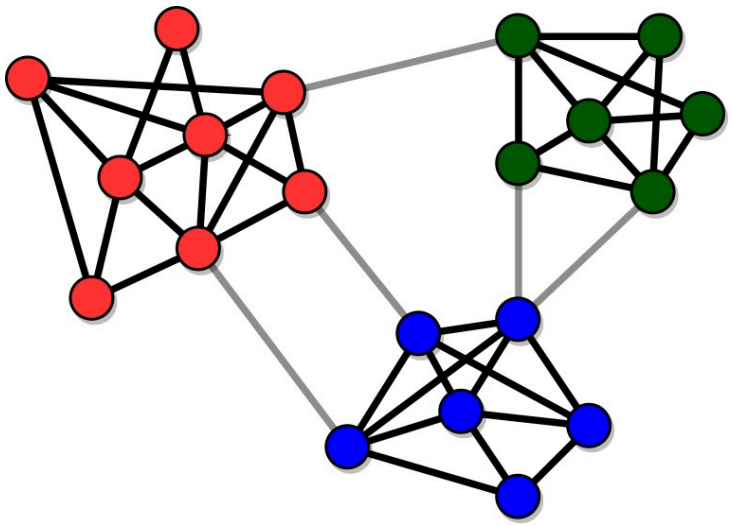
- A network can have:
  - similar global statistics
  - similar degree sequence
  - similar clustering
- and still differ drastically in how nodes are organized into groups

Community structure often captures:

- organization
- function
- redundancy
- coordination
- segregation

Communities as a structural picture

Communities are often imagined as **sets of nodes that are more connected to one another than to the rest of the network**.



Why we care about communities

- They summarize the network at an interpretable intermediate scale
- They suggest where information, influence, or resources may circulate more intensely
- They often correspond to organizational or functional units
- They help us move from descriptive graph statistics to structural explanation

Social example

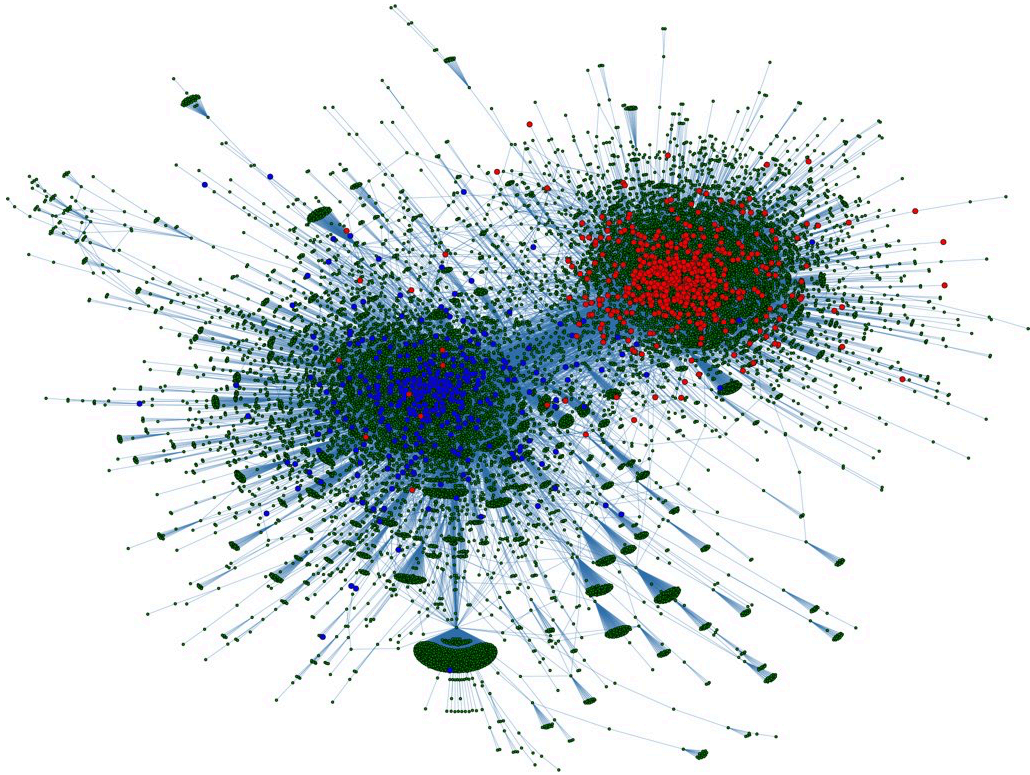
- In social networks, communities may correspond to:
  - friendship circles
  - political blocs
  - workplace teams
  - hobby groups
- But they do not always coincide perfectly with declared social categories

This is already a warning:

- the network structure and the metadata do not have to match exactly

Example: political homophily online

- Users with similar political preferences often connect more to one another than to politically distant users
- This can produce visible clusters in the follower graph
- But the cluster structure may reflect:
  - ideology
  - geography
  - language
  - activity level



Biological example

- In protein interaction networks, communities may correspond to:
  - functional modules
  - biochemical pathways
  - complexes
- The network is useful because it groups entities that often act together
- But again, the meaning of the group depends on the scientific question

Information-network example

- On the Web or in citation networks, communities may correspond to:
  - topics
  - scientific subfields
  - language domains
  - institutional ecosystems
- These are not always the same thing

So even before we talk about algorithms, we already face an interpretive issue:

- **which kind of grouping are we looking for?**

Infrastructure example

- In transportation or mobility networks, communities may reveal:
  - regional subsystems
  - mobility basins
  - administrative divisions
  - bottlenecks between subnetworks

This is useful for:

- planning
- resilience analysis
- intervention design

What do we really mean by a community?

First intuitive answer

- A community is a set of nodes with:
  - many links inside
  - relatively few links outside

This intuition is powerful, but it is still incomplete.

Why?

- How many internal links are "many"?
- Few compared to what?
- What if the group is sparse but still structurally coherent?
- What if nodes belong to more than one group?

The community problem is ill-defined

- There is **no single universally accepted definition** of community
- Different methods formalize different intuitions
- Different methods may return different partitions on the same graph
- Those partitions can all be defensible under different assumptions

This is not a bug in the field.

It is a consequence of the fact that "community" is a **family of mesoscale patterns**, not a single mathematical object.

A useful way to think about the problem

When we say "community", we are usually combining several ideas:

- **density**
- **cohesion**
- **separation**
- **connectedness**
- sometimes also:
  - similarity
  - function
  - flow persistence
  - statistical regularity

Different algorithms emphasize different subsets of these ideas.

Density hypothesis

- The most common intuition is the **density hypothesis**
- A community should be denser internally than externally

This works well for many social and information networks.

But density alone is not enough:

- two disconnected dense pieces should not usually be one community
- a tree-like but well-separated region may still be meaningful

Connectedness hypothesis

- A community should usually be a **connected subgraph**
- If a "community" is made of two disconnected pieces, we often suspect that something is wrong

Connectedness matters because a group should usually support:

- interaction
- flow
- cohesion
- interpretability

Separation hypothesis

- A community should be not only internally coherent
- It should also be reasonably separated from the rest of the network

So communities involve a tension between:

- being dense enough inside
- being distinct enough outside

This is why community detection is not reducible to just counting edges.

One network, several plausible partitions

- In real networks, there is often no unique "obvious" answer
- A network can support:
  - a coarse partition
  - a finer partition
  - an overlapping description
  - a hierarchical description

This is one reason why method choice matters so much.

Basic variables

Internal and external degree

For a node i in a community C:

- **internal degree k_i^{int}:**
  - number of neighbors of i that are also in C
- **external degree k_i^{ext}:**
  - number of neighbors of i outside C

These variables are local, node-level descriptors of how well the node is embedded in its assigned group.

Why these variables matter

- If k_i^{int} is large relative to k_i^{ext}, node i is strongly embedded in the community
- If k_i^{ext} is large, the node may:
  - sit near a boundary
  - connect multiple groups
  - have been assigned to the wrong group

So internal and external degree help us think about **membership strength**.

Community degree and volume

For a community C:

- the **community degree** or **volume** is

$$k_C = \sum_{i \in C} k_i$$

- the number of internal links is L_C

These aggregate quantities become important later when we define quality functions such as modularity.

Internal link density

For a community C with n_C nodes and L_C internal edges:

$$\rho_C = \frac{L_C}{n_C(n_C - 1)/2}$$

- $\rho_C = 1$ means a clique
- ρ_C close to 0 means very sparse internal structure

This captures one notion of cohesion, but not separation.

Boundary and external exposure

Besides internal density, a community is characterized by how many links leave it.

This suggests two complementary questions:

- How dense is the community internally?
- How strongly is it connected to the rest of the graph?

Community detection always balances these two dimensions.

Strong and weak communities

Strong community

A subgraph C is a **strong community** if every node satisfies:

$$k_i^{int} > k_i^{ext}$$

Interpretation:

- every single node is more connected inside the group than outside it

This is a strict requirement.

Why the strong definition is strict

- One boundary node can violate it
- One broker node can violate it
- One noisy or ambiguous node can violate it

So strong communities are conceptually clean, but often too restrictive for empirical data.

Weak community

A subgraph C is a **weak community** if:

$$\sum_{i \in C} k_i^{int} > \sum_{i \in C} k_i^{ext}$$

Interpretation:

- the group, taken as a whole, is more connected internally than externally

This allows some nodes to sit on the boundary.

Strong implies weak

If each node individually has more links inside than outside, then summing over nodes gives:

$$\sum_{i \in C} k_i^{int} > \sum_{i \in C} k_i^{ext}$$

So:

- every strong community is also a weak community

But the converse is false.

Weak does not imply strong

A community may be weak overall even if:

- several nodes are strongly embedded
- one or two nodes are weakly embedded or ambiguous

This is often what happens in real networks:

- clear cores
- fuzzy boundaries

That is one reason why weak definitions are often more empirically plausible.

But even strong/weak definitions are not enough

These definitions compare a subgraph with **the rest of the network as a whole**.

That can be unsatisfactory.

Why?

- In a network with many groups, it often makes more sense to compare a node or subgraph to **specific other communities**, not to the aggregate outside world.

Less stringent reformulations

Alternative formulations compare the inside of a community to the **largest competing outside group** rather than to the rest of the graph combined.

This matters because:

- "outside" is not homogeneous
- groups compete with one another asymmetrically
- boundaries are often local, not global

What we learn from these definitions

- They are useful because they formalize core intuitions
- But they are not complete solutions
- They tell us that community detection is about:
  - cohesion
  - separation
  - local ambiguity
  - boundary structure

That is exactly why we need multiple algorithmic families.

Partitions, overlap, and hierarchy

Partition

A **partition** of the node set is a collection of groups such that:

- every node belongs to exactly one group
- no node belongs to two groups
- together, the groups cover all nodes

Many classical algorithms assume partitions.

Why partitions are convenient

- They simplify optimization
- They simplify comparison
- They simplify visualization
- They match some applications well

But they also impose a strong assumption:

- each node has exactly one mesoscale identity

That assumption is often too rigid.

Bell numbers

- The number of possible partitions of N nodes is the **Bell number** B_N
- Bell numbers grow extremely fast

For community detection, the message is not the exact value.

The message is:

- the search space is enormous
- brute-force search is impossible except for very small networks

Computational implication

Because the space of partitions is huge:

- we cannot enumerate all possibilities
- we need heuristics, relaxations, or statistical models
- each method searches the space differently

This is the computational reason why community detection depends on algorithmic design.

Overlapping communities

In many systems, a node can naturally belong to more than one group.

Examples:

- a person belongs to family, work, and hobby circles
- a paper belongs to multiple topics
- a protein may participate in multiple functional modules

So a partition is sometimes too restrictive.

Why overlap matters conceptually

- Overlap is not a technical corner case
- It reflects the fact that mesoscale structure can be **multi-membership**

If we force a partition where overlap is natural, we may:

- distort the groups
- misclassify boundary nodes
- overstate the clarity of the network structure

Hierarchical communities

Communities can also be nested across scales.

For example:

- research groups inside departments
- departments inside faculties
- regional subnetworks inside national systems

This means that there may be no single privileged scale.

Scale matters

Two partitions can both be meaningful:

- one coarse and interpretable
- one finer and more detailed

This is a central point:

- community detection is often a **multi-resolution** problem

Later, this will return when we discuss:

- modularity limits
- Louvain/Leiden hierarchies
- benchmarks and validation

Mid-lecture recap

- A community is not defined by one universal rule
- Density, connectedness, and separation all matter
- Strong and weak communities capture useful but partial intuitions
- Partitions are convenient but restrictive
- Overlap and hierarchy are not side issues; they are central features of mesoscopic structure

Related problems

Community detection versus graph partitioning

These are related, but not identical.

- **Graph partitioning** usually starts from an optimization objective such as:
  - minimizing cut size
  - balancing group sizes
- **Community detection** asks whether the resulting groups correspond to meaningful mesoscale structure

Partitioning is often an engineering objective.

Community detection is usually an interpretive structural objective.

Why graph partitioning is not enough

Suppose we minimize the number of edges between two groups.

This may produce:

- balanced halves
- low cut cost

But that does not guarantee:

- dense internal groups
- meaningful communities
- realistic social or functional modules

So graph partitioning gives an important baseline, not a full answer.

Kernighan-Lin belongs to this family

- The Kernighan-Lin algorithm is a classic graph partitioning heuristic
- It is historically important
- It teaches us how local swaps can improve a cut objective

But its goal is not yet "find natural communities in a network" in the broader network-science sense.

We will return to it in `NS13` as a baseline.

Community detection versus clustering

- In data clustering, we often start from:
  - feature vectors
  - distances
  - similarities
- In community detection, we often start from:
  - the graph topology itself

These are related because a graph can induce a similarity notion.

But the tasks are not identical.

Hierarchical clustering

Hierarchical clustering is important because:

- it produces a dendrogram
- it makes the multi-scale aspect visible
- it provides a useful bridge between similarity-based grouping and community analysis

But similarity-based clusters need not correspond to dense communities in the graph.

Structural equivalence is not the same as community

Two nodes can be **structurally similar** if they connect to the same kinds of nodes.

But they may:

- belong to different communities
- play the same role in different parts of the network

So similarity-based clustering can reveal:

- roles
- positions
- equivalence classes

not necessarily communities.

Communities versus roles

- **Communities** group nodes that are tightly or persistently linked to one another
- **Roles** group nodes that occupy similar structural positions

These are different mesoscale questions.

A network-science course should keep them separate conceptually, even if some algorithms blur the line.

What kinds of outputs can methods return?

Different methods may return:

- a hard partition
- an overlapping cover
- a hierarchy
- a score for each partition
- a probabilistic assignment
- a fitted generative model

This is another reason why community detection is not one single problem.

A map of method families

Family 1: divisive and cut-based methods

- Remove or cut strategically important edges
- Or optimize partitioning objectives
- Good for:
  - intuition
  - small networks
  - bridge-based reasoning

Examples:

- Kernighan-Lin
- Girvan-Newman

Family 2: quality-function optimization

- Define a score for partitions
- Search for partitions that maximize that score

The most famous example is:

- **modularity**

This family will dominate `NS13` and part of `NS14`.

Family 3: flow-based and information-theoretic methods

- View the network through random walks or flows
- Seek communities that trap or compress movement

The canonical example is:

- **Infomap**

These methods do not optimize the same objective as modularity.

Family 4: propagation methods

- Start from local labels
- Let communities emerge from local agreement

These methods are often:

- simple
- fast
- scalable

But they may also be unstable.

Family 5: generative and model-based methods

- Assume the network was generated by an underlying statistical model
- Infer the hidden mesoscale structure that best explains the observed edges

The central family here is:

- **stochastic block models**

This is a different philosophy from pure optimization.

What changes from one family to another?

- the definition of community
- the optimization target
- the role of randomness
- the type of output
- the way we validate the result

So the next lectures are not just "more algorithms".

They are different answers to the question:

- **what kind of mesoscale structure are we trying to detect?**

Where we go next

- `NS13` :
  - partitioning baselines
  - bridge removal
  - modularity
  - spectral ideas
- `NS14` :
  - scalable modern methods
  - Louvain
  - Leiden
  - Infomap
  - label propagation
- `NS15` :
  - stochastic block models
  - benchmarks
  - validation
  - interpretation

Main takeaway

- Community detection is a problem about **mesoscopic organization**
- There is no single perfect definition of community
- Different methods encode different structural assumptions
- Before choosing an algorithm, we need to know what kind of groups we are looking for

That is the conceptual foundation for the rest of the block.

Reading material

- **Menczer, Fortunato, Davis.** *A First Course in Network Science*, Chapter 6
- **Barabasi.** *Network Science*, Chapter 9
- **Fortunato, S., & Hric, D. (2016).** Community detection in networks: A user guide. *Physics Reports*, 659, 1-44
- **Borgatti, Everett, & Shirey (1990).** LS sets, lambda sets and other cohesive subsets. *Social Networks*

Q&A